📘 1. FRONTEND REPORT (React + Vite)

🔹 Architecture Overview

The frontend of the Leave Management System is developed using React with Vite, following a component-based architecture. This approach ensures modularity, reusability, and scalability. Each UI element is encapsulated into independent components, making the application easier to maintain and extend.

🔹 Folder Structure

The project is structured to separate concerns:

components/ → Reusable UI components (buttons, forms, cards)

pages/ → Main screens like Login, Dashboard, LeaveList, ApplyLeave

services/ → Handles API calls using Axios

utils/ → Helper functions for formatting and validation

This separation improves readability and maintainability.

🔹 Key Components

1. Login Component

Handles user authentication

Sends credentials to backend

Stores JWT token on successful login

2. Dashboard

Displays user-specific data (leave balance, achievements)

Dynamically updates based on role

3. LeaveList

Shows leave records

Filters data based on role:

Employee → own leaves

Manager → team leaves

Admin → all leaves

4. ApplyLeave

Form for submitting leave requests

Includes document upload feature

Performs validation before submission

- Routing System

Implemented using React Router:

/login → authentication page

/dashboard → user overview

/leave/list → leave records

/leave/apply → apply leave

Routing ensures smooth navigation without full page reloads.

- State Management

useState → handles form inputs, UI states

useEffect → fetches data from backend

LocalStorage → stores JWT token

This ensures efficient state updates and persistence.

- API Integration

Axios is used for API communication:

JavaScript

```
const API = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL
});
```

Centralized API handling

Cleaner and reusable requests

- Validation & Error Handling

Inline error messages near form fields

No popup/toast spam (improved UX)

Prevents invalid submissions

- UI/UX Design Decisions

Role-based UI rendering

Minimal and clean design

Conditional rendering (buttons shown only when needed)

Improved document UI (View → then actions)

📗 2. BACKEND REPORT (Node.js + Express)

🔹 Architecture

Backend follows MVC architecture:

Models → database structure

Controllers → business logic

Routes → API endpoints

This ensures clean separation of concerns.

🔹 Folder Structure

routes/ → defines endpoints

controllers/ → handles logic

models/ → MongoDB schemas

middleware/ → authentication, error handling

🔹 Authentication (JWT)

User logs in → credentials verified

JWT token generated

Token sent to frontend

Used for protected routes

Example:

JavaScript

```
jwt.sign(payload, secret, { expiresIn: '7d' });
```

🔹 Role-Based Access Control

Role

Access

Employee

Apply/view own leaves

Manager

Manage team leaves

Admin

Full access

Implemented using middleware and conditional queries.

- Middleware

1. Authentication Middleware

Verifies JWT token

Attaches user info to request

2. CORS Middleware

Allows frontend communication

3. Error Handler

Centralized error handling

4. Multer (File Upload)

Handles document uploads

Stores files in /uploads

- Document Handling

Files saved locally in /uploads

URL generated:

Plain text

https://backend-domain/uploads/file.pdf

Stored in database as string

- API Endpoints

Method

Endpoint

Description

POST

/api/auth/login

Login

POST

/api/leaves

Apply leave

GET

/api/leaves

Get leaves

PATCH

/api/leaves/:id

Approve/Reject

🔹 Validation Logic

Prevent invalid dates

Prevent exceeding leave limits

Ensure required fields

🔹 Security

JWT authentication

Protected routes

Input validation

📕 3. DATABASE REPORT (MongoDB)

🔹 Database Design

MongoDB is used as a NoSQL database, allowing flexible schema design.

🔹 Collections

User Collection

name

email

password

role

department

Leave Collection

employeeId

leaveType

fromDate

toDate

duration

status

document (URL)

documentStatus

- Relationships

One user → multiple leaves

Manager → filters based on department

- Data Flow

User submits leave

Stored in database

Manager retrieves and updates

- Operations

Create → Apply leave

Read → Fetch leaves

Update → Approve/Reject

Delete → Cancel

- Validation

Required fields enforced

Data types validated

Status transitions controlled

🔗 4. INTEGRATION REPORT

🔹 Communication Flow

Frontend communicates with backend using REST APIs via Axios.

🔹 Environment Variables

Frontend:

Environment

VITE_API_BASE_URL=https://backend-url/api

Backend:

Environment

MONGO_URI=...

JWT_SECRET=...

🔹 Authentication Flow

User logs in

Backend returns JWT

Token stored in frontend

Sent in headers for future requests

🔹 Role-Based Data Handling

Backend filters data

Frontend renders accordingly

🔹 Error Handling

Backend sends structured errors

Frontend displays inline messages

🔹 CORS Handling

Backend allows requests from frontend domain:

JavaScript

origin: ['https://vercel-app-url']

🚀 5. DEPLOYMENT REPORT

- Backend Deployment (Render)

Platform: Render

Build: npm install

Start: node server.js

Environment Variables

Environment

MONGO_URI=...

JWT_SECRET=...

JWT_EXPIRE=7d

- Frontend Deployment (Vercel)

Framework: Vite

Build: npm run build

Output: dist

- Integration After Deployment

Frontend connects to backend using ENV

API calls updated to deployed URL

- Issues Faced & Fixes

1. Login 404 Error

Cause: wrong endpoint

Fix: correct /api/auth/login

2. CORS Error

Fix: allow Vercel URL in backend

3. Document 404

Fix: store full backend URL

4. MongoDB Connection Error

Cause: missing .env

Fix: add MONGO_URI

BACKEND REPORT (Node.js + Express) — DETAILED

🔹 Overall Architecture

The backend is built using Node.js with Express.js, following a modular MVC (Model-View-Controller) architecture. This design separates:

Models → Define data structure (MongoDB schemas)

Controllers → Contain business logic

Routes → Define API endpoints and map them to controllers

Middleware → Handle cross-cutting concerns (authentication, errors, CORS)

👉 This separation ensures:

Better scalability

Easy debugging

Clean code structure

🔹 Request Flow (VERY IMPORTANT)

When a request is made:

Plain text

Frontend → Route → Middleware → Controller → Database → Response → Frontend

Example (Apply Leave):

User submits form

Request hits /api/leaves route

Auth middleware verifies JWT

Controller processes data

Data saved in MongoDB

Response sent back

🔹 Controllers (Core Logic)

Controllers contain the actual functionality of the system.

Example: Apply Leave Logic

Extract data from request

Validate leave type, dates

Check leave balance

Handle file upload (if any)

Generate document URL

Save leave in DB

👉 Important validation:

Prevent exceeding leave limits

Ensure required fields

🔹 Authentication System (JWT)

Flow:

User logs in

Backend verifies credentials

JWT token generated

JavaScript

jwt.sign(payload, secret, { expiresIn: '7d' })

Token sent to frontend

Frontend sends token in headers

Middleware Usage

Auth middleware:

Plain text

Checks token → verifies → attaches user to req.user

👉 Without this:

Routes remain unprotected ❌

🔹 Role-Based Access Control (RBAC)

The system enforces access control based on roles:

Employee

Apply leave

View own leaves

Manager

View team leaves

Approve/reject

Admin

Full control

👉 Implemented using:

JavaScript

```
if (user.role === "Manager") { ... }
```

🔹 File Upload System

Handled using Multer middleware

Process:

File uploaded from frontend

Stored in /uploads folder

Filename generated uniquely

URL created:

Plain text

```
https://backend-domain/uploads/file.pdf
```

Stored in DB

🔹 Error Handling

All errors handled centrally

Returns structured responses:

JSON

```
{
  "message": "Leave limit exceeded"
}
```

👉 Helps frontend display clean messages

🔹 Security Measures

JWT authentication

Protected routes

Input validation

Controlled access

📕 DATABASE REPORT (MongoDB + Mongoose) — DETAILED

🔹 Why MongoDB?

MongoDB is a NoSQL database:

Flexible schema

JSON-like documents

Easy integration with Node.js

🔹 Schema Design

User Schema

Stores authentication and role info:

Plain text

name, email, password, role, department

Leave Schema

Stores leave details:

Plain text

employeeId

leaveType

fromDate

toDate

duration

status

document (URL)

documentStatus

🔹 Relationships

One User → Many Leaves

Manager → filters by department

👉 No joins (NoSQL), but handled logically

🔹 Data Flow

User submits leave

Stored in Leave collection

Manager queries leaves

Updates status

🔹 Query Operations

Create

JavaScript

Leave.create(…)

Read

JavaScript

Leave.find(…)

Update

JavaScript

leave.status = "Approved"

Delete

JavaScript

leave.status = "Cancelled"

🔹 Aggregation (IMPORTANT)

Used for:

Calculating leave balance

Counting leaves

Example:

JavaScript

$group → sum duration

🔹 Validation

Required fields

Correct data types

Controlled status updates

🔹 Data Integrity

Ensures:

No invalid leaves

No negative balance

Consistent state

🔗 INTEGRATION REPORT (Frontend + Backend) — DETAILED

🔹 Communication Architecture

Frontend and backend communicate via:

Plain text

REST APIs (HTTP Requests)

🔹 Axios Configuration

Frontend uses:

JavaScript

```
axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL
})
```

🔹 API Flow Example (Login)

User enters credentials

Frontend sends:

Plain text

POST /api/auth/login

Backend verifies

Returns JWT

Stored in frontend

- Token Usage

Frontend sends token in headers:

Plain text

Authorization: Bearer <token>

Backend verifies before allowing access

- Role-Based Data Handling

Backend filters data:

Employee → own

Manager → team

Admin → all

Frontend just displays

- Error Handling Flow

Backend sends error:

JSON

{ "message": "Leave limit exceeded" }

Frontend displays inline message

👉 Clean UX

- CORS Handling

Since frontend & backend are different domains:

Plain text

Vercel → Frontend

Render → Backend

Backend allows:

JavaScript

origin: ['https://vercel-url']

- Deployment Integration Issues

1. 404 Errors

Wrong API route

Fixed by correct endpoint

2. Document Issues

Frontend tried to open /uploads

Fixed by using full backend URL

3. Environment Variables

Used for flexibility

Prevent hardcoding

- Final Integration Flow

Plain text

User → Frontend → API → Backend → DB → Response → UI update